Core Gym Club

Booking System

Project Documentation

A modern, responsive gym session booking platform built for Core Gym Club AB, Haninge.

1. Project Overview

Background

Core Gym Club AB is a gym chain based in Haninge that offers group training, personal training, and open gym sessions. The company requested a digital booking system that allows members to book gym sessions, view schedules, and manage memberships.

The goal of this project is to build a modern, responsive, and user-friendly booking platform that serves members and administrators efficiently.

2. Project Goals

- Develop a modern gym booking system
- Allow users to register and log in
- Allow members to view and book training sessions
- Allow administrators to manage gym classes
- Deliver a responsive application for desktop and mobile devices

3. MVP Scope

User Features

- Register account
- Login / logout
- View upcoming training sessions
- Book a training session
- Cancel a booking
- View profile information

Admin Features

- Create training sessions
- Edit training sessions
- Delete training sessions
- Manage schedules

4. Agile Workflow

The development team used Agile methodology throughout the project. **Trello** was used for sprint planning and task management.

Workflow Columns

- Backlog
- To Do
- In Progress

- Testing
- Done

The team delivered a runnable MVP version every week, ensuring continuous progress and early feedback incorporation.

5. Git Workflow

Git and GitHub were used for version control with a feature-branch strategy.

Branch Examples

- `feature/login`
- `feature/booking-system`
- `feature/admin-dashboard`
- `feature/profile-page`

All completed features were merged into the **main** branch using pull requests, ensuring code review before integration.

6. User Stories

ID	Role	I want to…	So that…
US-01	User	create an account	access the booking system
US-02	User	log in	manage my bookings
US-03	User	log out	keep my account secure
US-04	Member	view upcoming gym classes	choose a suitable session
US-05	Member	book a class	reserve my spot
US-06	Member	cancel a booking	free my reserved spot
US-07	Member	view my upcoming bookings	manage my schedule
US-08	Admin	create training sessions	members can book them
US-09	Admin	edit training sessions	incorrect info can be updated
US-10	Admin	delete sessions	cancelled classes are removed

7. Activity Diagram – Booking Flow

The activity diagram below models the step-by-step flow a member follows to book a gym session, including the system decision point that checks spot availability.

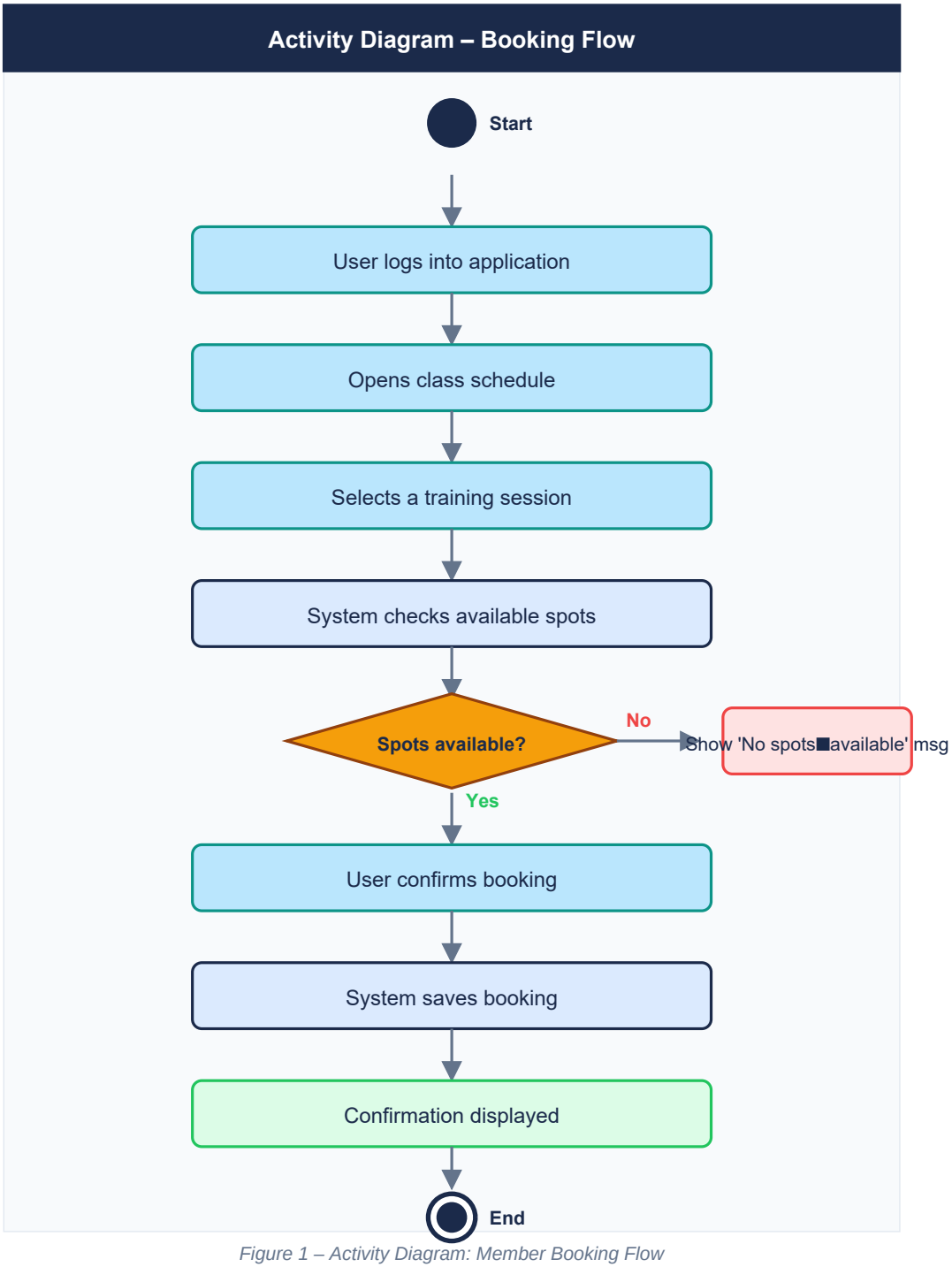

Figure 1 – Activity Diagram: Member Booking Flow

Diagram Walk-Through

Start: The flow begins when a registered member initiates a session booking.

Login: The user authenticates into the application using ASP.NET Authentication.

Open Schedule: The member navigates to the class schedule page.

Select Session: The member picks a desired training session.

Availability Check: The system queries ASP.NET Authentication to verify remaining spots.

Decision – Spots Available?

• *Yes:* The user confirms the booking. The system persists the record in ASP.NET Authentication and displays a confirmation message.

• *No:* A 'No spots available' message is shown and the flow ends without a booking.

End: The booking process completes.

8. Sequence Diagram – Booking Flow

The sequence diagram illustrates the chronological message exchange between the **User**, **Frontend** (React), **Backend** (ASP.NET Core Web API), and the **Database** (ASP.NET Authentication) during a booking operation.

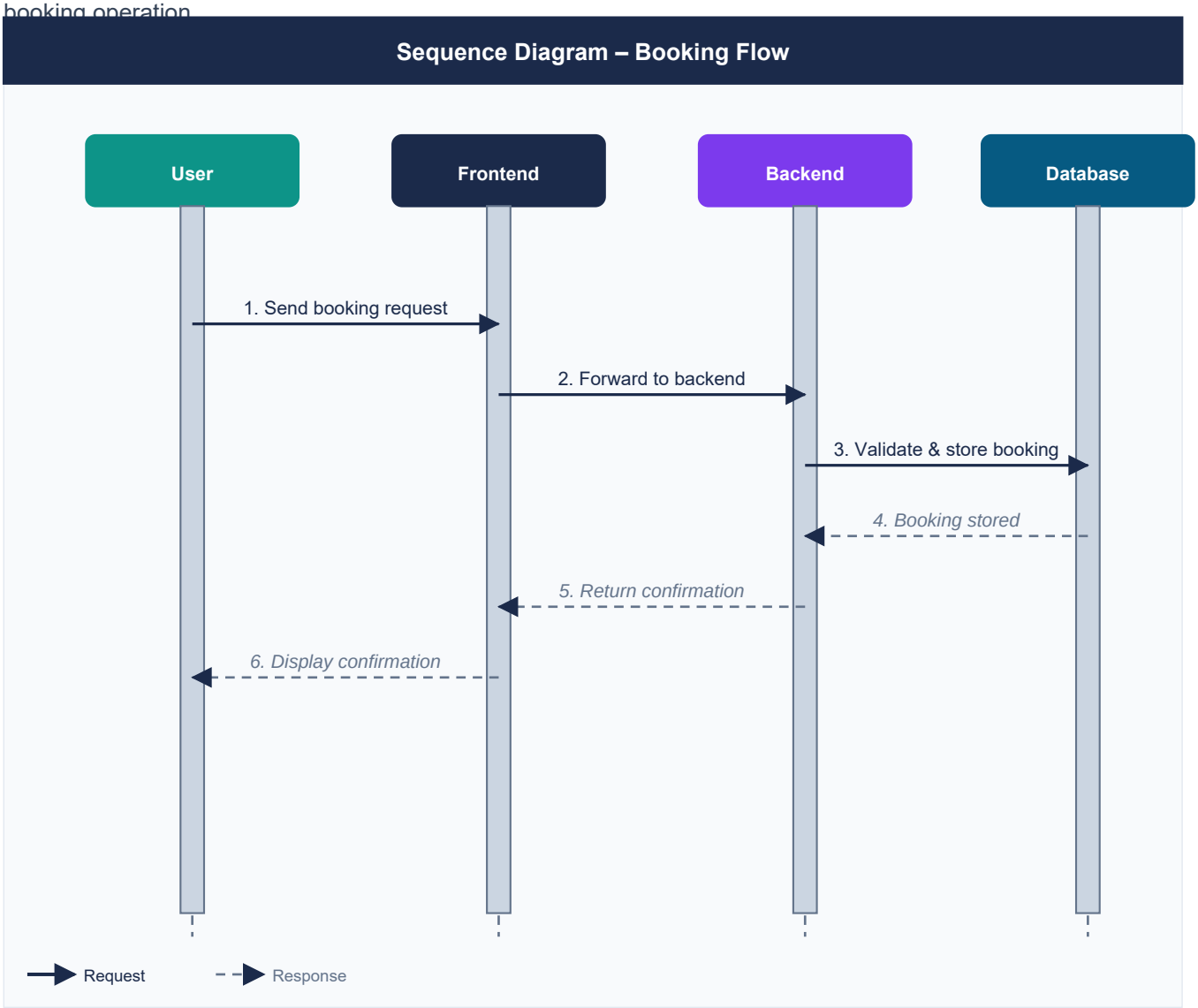

Figure 2 – Sequence Diagram: Member Booking Flow

Message Flow Description

1. **Send booking request:** User taps 'Book' on the frontend UI.

2. **Forward to backend:** React sends an API/SDK call to Firebase.

3. **Validate & store:** Backend checks spot count and writes the booking to ASP.NET Authentication.
4. **Booking stored:** ASP.NET Authentication confirms the write operation (response).
5. **Return confirmation:** Backend propagates the success response to the frontend.
6. **Display confirmation:** The UI renders a success message to the user.

9. User Authentication Module

Authentication is handled entirely by **ASP.NET Authentication**. The module supports email/password registration and login, session persistence, and secure logout. A JWT token is issued upon successful login and used for all subsequent authenticated requests.

9.1 Activity Diagram – User Authentication

The diagram below models the login and credential-validation flow, including the branching logic for invalid credentials.

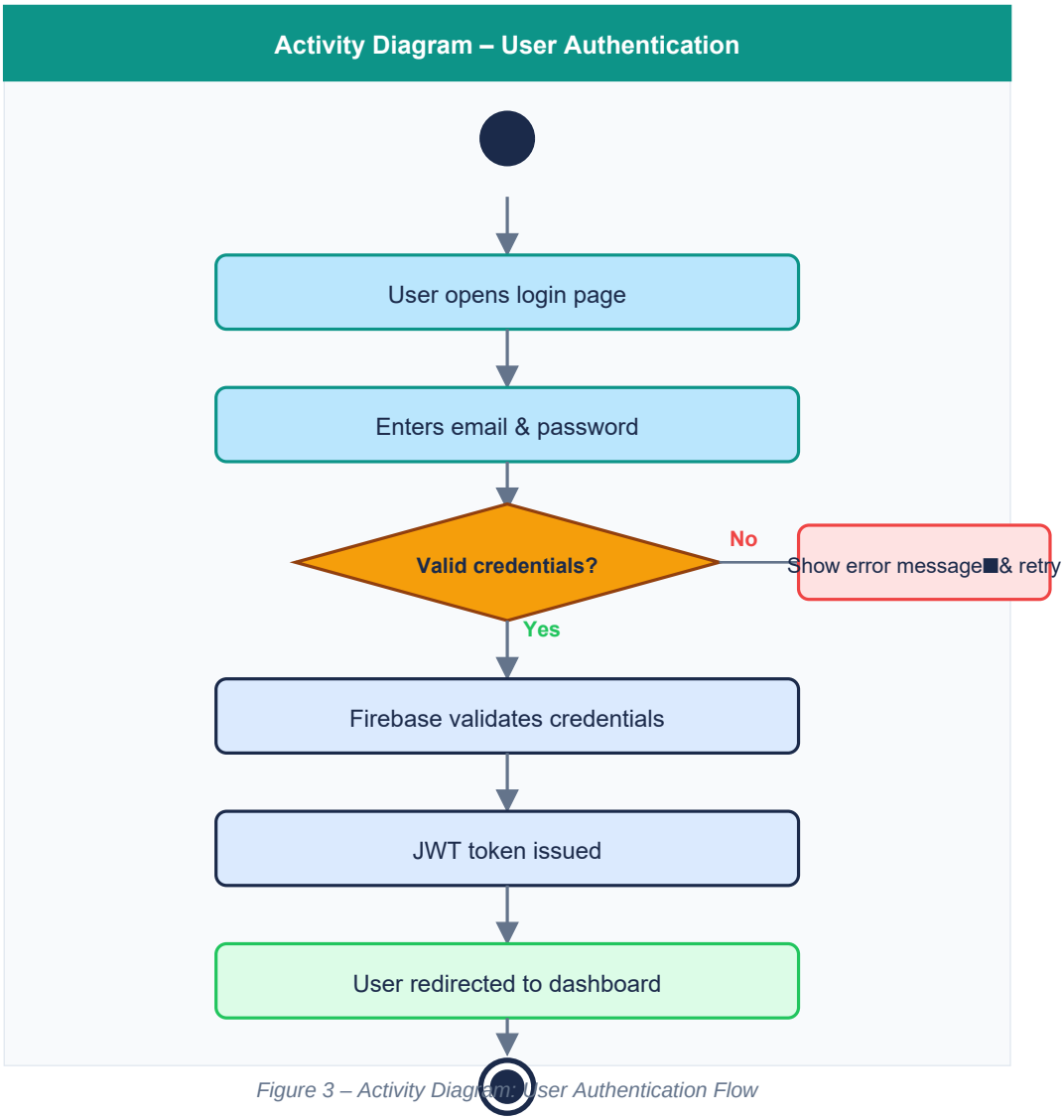

Figure 3 – Activity Diagram: User Authentication Flow

9.2 Sequence Diagram – User Authentication

This diagram shows the full message exchange from login form submission through Firebase token issuance to dashboard redirect, as well as the logout sequence.

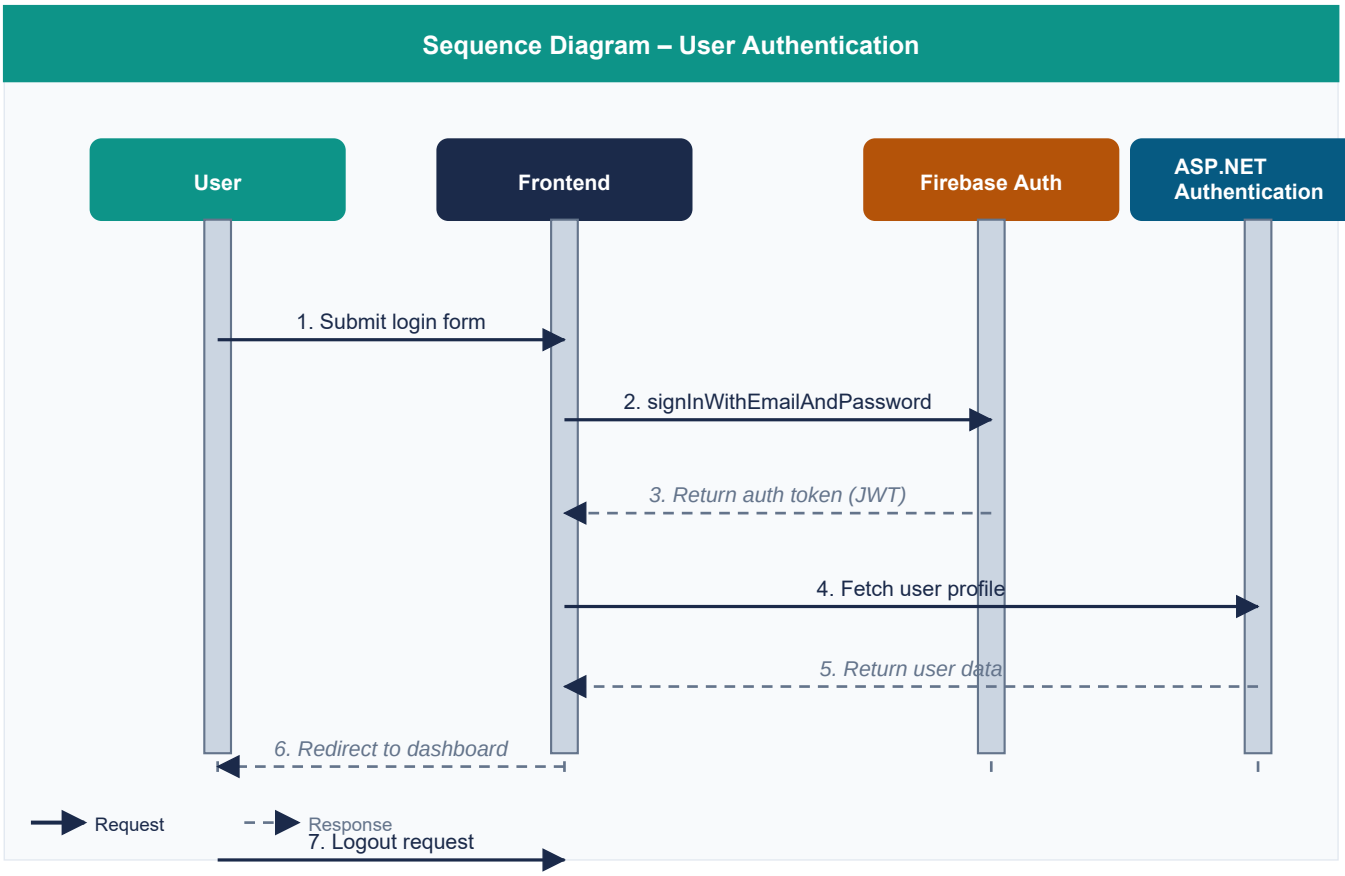

Figure 4 – Sequence Diagram: User Authentication Flow

Authentication Walk-Through

1. User opens login page and submits email/password credentials.
2. Frontend calls `signInWithEmailAndPassword()` via Firebase SDK.
3. Firebase Auth validates credentials and returns a JWT auth token.
4. Frontend fetches the user profile document from ASP.NET Authentication.
5. User is redirected to the member dashboard.
6. On logout, `signOut()` is called, clearing the session.

10. Admin Session Management Module

The admin module provides authenticated administrators with full control over training sessions: creating new sessions, editing existing ones, and deleting cancelled sessions. All changes are persisted in real-time to ASP.NET Authentication.

10.1 Activity Diagram – Admin Session Management

The branching activity diagram below shows the three paths an admin can take after viewing the session list: Create, Edit, or Delete.

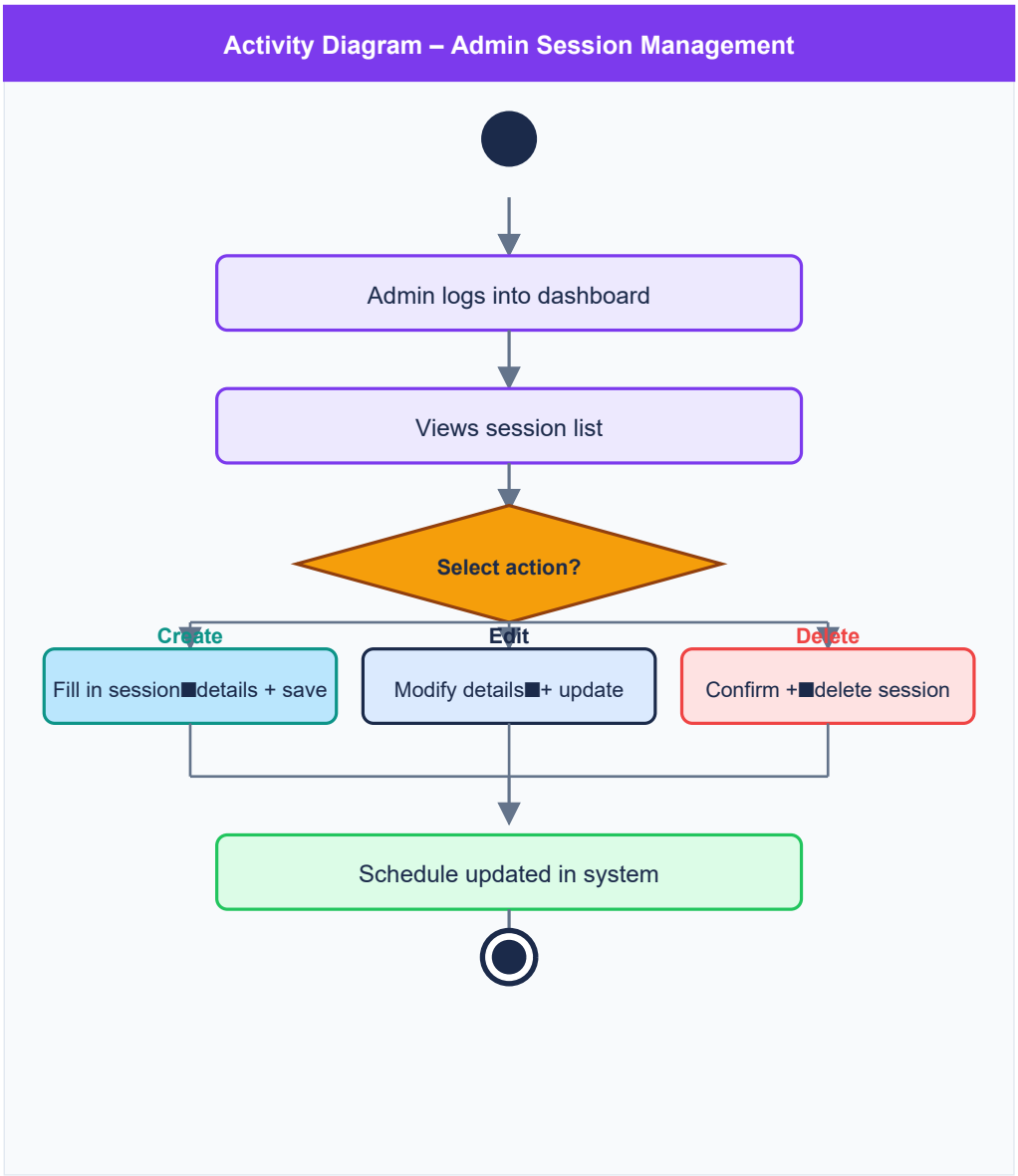

Figure 5 – Activity Diagram: Admin Session Management

10.2 Sequence Diagram – Admin Session Management

This diagram captures the full interaction between the admin, the frontend, the backend layer, and ASP.NET Authentication — from loading the session list to confirming a create/edit/delete operation.

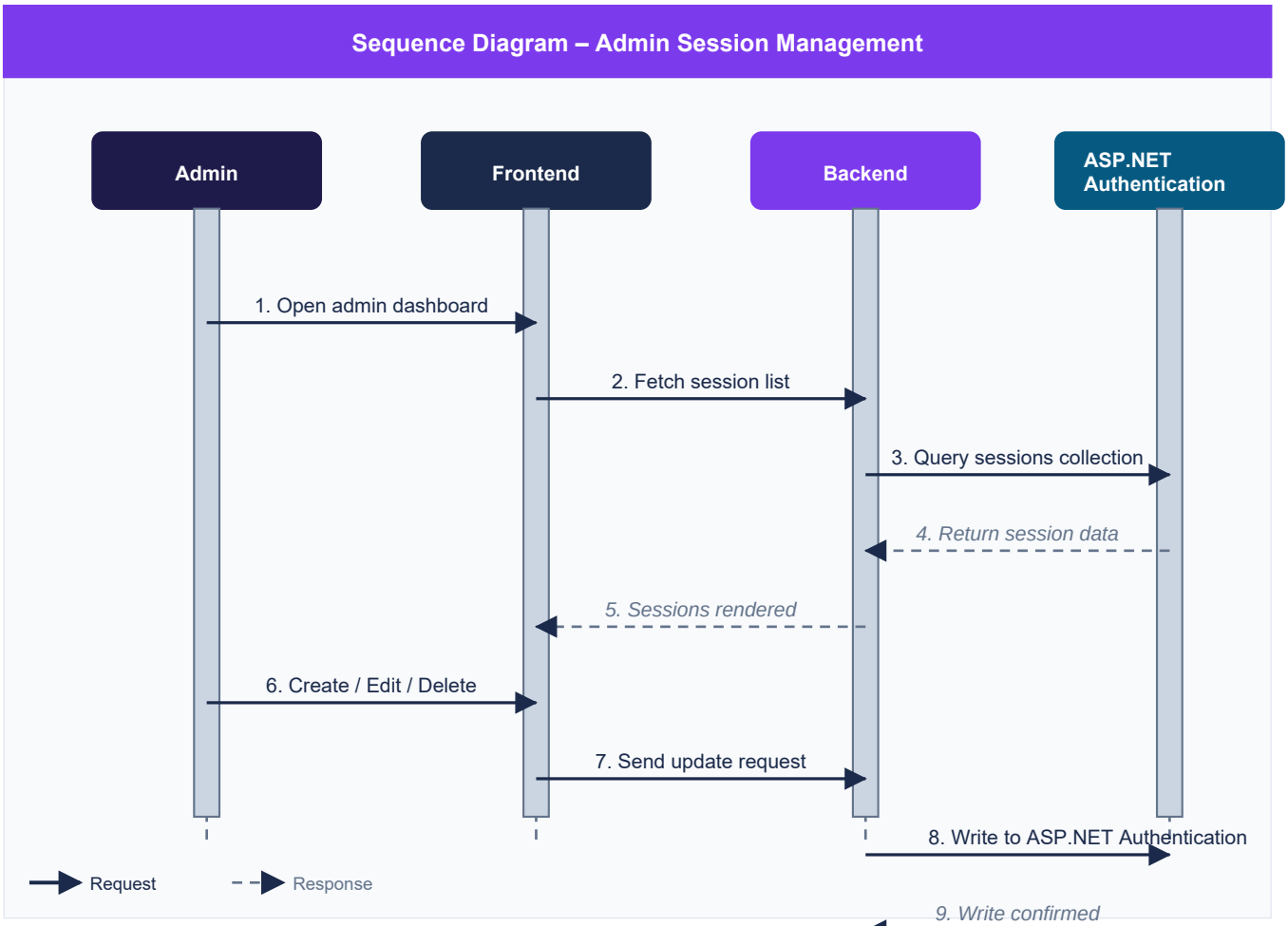

Figure 6 – Sequence Diagram: Admin Session Management

Admin Flow Walk-Through

1. Admin opens the admin dashboard (authenticated route).
2. Frontend fetches the current session list from ASP.NET Authentication via the backend.
3. Sessions are rendered in the admin UI.
4. Admin selects Create, Edit, or Delete.
5. The update request is sent to the backend.
6. Backend writes the change to the ASP.NET Authentication sessions collection.
7. ASP.NET Authentication confirms the write; the admin receives a success notification.

11. Technical Stack

Layer	Technology
Frontend	React
Backend	ASP.NET Core Web API
Database	SQLite
ORM	Entity Framework Core
Authentication	ASP.NET
API Documentation /Testing	Swagger
Version Control	Git + GitHub
Branch Management	Feature branches + Pull Requests
Project Management	Trello
Design	Responsive Web Design
Development Tool	Visual Studio + VS Code
Programming Languages	C#, JavaScript, HTML, CSS, React
Deployment / Hosting	GitHub

12. Responsive Design

The application was designed to work seamlessly across all device types:

- Mobile devices (iOS & Android via React responsive web design)
- Tablets
- Desktop browsers (React web)

Responsive layouts were implemented to improve usability and maintain a consistent experience across all screen sizes.

13. Challenges Faced

Challenge	Resolution
Firebase environment configuration	Improved Firebase initialization scripts
Production build crashes in Swagger	Careful production build testing & fallback configs
Managing authentication persistence	Adopted Firebase Auth persistence settings
Offline states & API failures	Added error handling and fallback configurations

14. Weekly MVP Progress

Sprint	Theme	Key Deliverables
Week 1	Foundation	• Project setup and scaffolding • ASP.NET Core Web API project configuration • Authentication system (register/login/logout)

Week 2	Core Booking	• Booking system implementation • Session listing and filtering • User profile features
Week 3	Admin & Polish	• Admin dashboard • Create / edit / delete sessions • Responsive layout improvements
Week 4	Delivery	• End-to-end testing and bug fixing • Final deployment via GitHub • Project documentation

15. Reflections

The project deepened the team's understanding of modern full-stack mobile/web development. Key learning outcomes included:

- Agile development and sprint management with Trello
- Git workflow: feature branches, pull requests, and code review
- ASP.NET Core Web API, Entity Framework Core och SQLite: Auth, ASP.NET Authentication, real-time updates
- Cross-platform responsive design for mobile and web
- Debugging and testing production builds on Swagger
- MVP planning and iterative feature prioritization

The team also reinforced the importance of:

- Continuous testing throughout development
- Thorough documentation
- Disciplined version control
- Incremental MVP delivery for fast feedback loops

16. Conclusion

The project successfully delivered a working MVP booking system for Core Gym Club AB. The application was built using Agile methods, Git workflows, and weekly MVP deliveries in accordance with the assignment requirements.

Members can:

- Register and log in securely
- View all available gym sessions
- Book and cancel training sessions
- View and manage their upcoming schedule

Administrators can:

- Create, edit, and delete training sessions
- Update and manage the full class schedule